Architecture and Mathematical Specification of the Agent-Native Programming Paradigm: State Separation, FFI, and the Bridge Layer

1. Introduction: The Categorical Imperative in Agentic Systems

The prevailing paradigms of software engineering are inextricably bound to human cognitive limitations. Legacy programming languages—such as C++, Python, and Rust—were evolutionarily optimized for human visual hierarchy, limited short-term working memory, and sequential lexical parsing. These languages treat code as flat strings of text, a medium that is inherently hostile to the high-dimensional, continuous tensor spaces natively inhabited by Large Language Models (LLMs). When autonomous AI agents are forced to interface with these rigid, one-dimensional lexical environments, they inevitably falter. They struggle with implicit control flow, fail to maintain loop-state tracking over extended context horizons, and hallucinate under the weight of imperative mutability constraints.

To resolve this profound impedance mismatch, it is imperative to abandon human-readable syntax in its entirety. The agent-native programming paradigm proposes a revolutionary environment where the LLM functions not as a typist emitting error-prone syntax, but as an orchestrator of pure, mathematically verifiable logic graphs.[1] This architecture treats code logic as a semantic intermediate representation composed of categorical objects, morphisms, paths, and types, grounding the execution entirely in Homotopy Type Theory (HoTT).[1] By embracing Typed Tokenization, the system parses natural language intent directly into these categorical structures before they are embedded into vectors, avoiding the ambiguity of lexical parsing entirely.[1]

This comprehensive specification encompasses both Phase 1 and Phase 2 of the paradigm's design. It first explicitly resolves the "State vs. Logic" conflation that plagues contemporary agentic systems by formalizing the structural separation of an immutable, graph-based codebase from an ephemeral, massively concurrent dynamic runtime. It subsequently establishes a secure Foreign Function Interface (FFI) utilizing Information-Flow Control (IFC) to interact safely with legacy systems. Finally, it defines the Phase 2 Bridge Layer—a highly constrained decoding grammar that forces autoregressive transformers to output valid Verifiable Programmatic Intermediate Representation (VPIR) nodes—and details the exact mathematical translation pipeline from these nodes into HoTT morphisms and SMT constraints.

2. The Core Architecture: The State vs. Logic Dichotomy

The central vulnerability of current LLM-driven applications, such as standard code interpreters, is the epistemological intermingling of static application logic with dynamic runtime state within the same context window. When an LLM evaluates a standard Python script while simultaneously tracking the values of instantiated variables, the boundary between immutable instruction and ephemeral data collapses. This collapse leads to context degradation, logical inconsistencies, and a severe susceptibility to prompt injection attacks.[3]

The agent-native architecture enforces a strict bipartite ontology, completely decoupling the rules of the system from the data flowing through it. The immutable codebase is parsed and stored as a non-Euclidean Tree-sitter Deterministic Knowledge Base (DKB), representing the pure platonic relationships between operational nodes.[4] Conversely, the ephemeral runtime state is executed via Dataflow Process Networks (DPN), where state is represented as discrete tokens flowing through unidirectional, non-blocking FIFO channels between stateless actor nodes.[6]

This architectural separation maps optimally to the realities of modern AI hardware topologies. The static logic graph resides in unified system memory, continuously queried via Geometric Deep Learning (GNNs) to provide structural context to the LLM. Simultaneously, the dynamic memory tokens traverse parallelized GPU edge clusters via zero-copy memory transfers, orchestrated by the LLM without ever requiring the model to memorize the state itself.

Architectural Component	**Legacy Paradigm (Python/C++)**	**Agent-Native Paradigm**	**Hardware Mapping**
Logic Representation	Flat lexical text files	Immutable Tree-sitter DKB Graph	Unified System Memory
Execution Model	Imperative, synchronous loops	Dataflow Process Networks (DPN)	Distributed GPU Clusters
State Tracking	In-place variable mutation	Unidirectional FIFO token streams	Zero-copy memory fabrics

Verification	Unit tests, runtime exceptions	Mechanically verified VPIR nodes	Z3 SMT Solvers
LLM Role	Syntax typist	Graph orchestrator	Autoregressive decoding

3. The Immutable Codebase: Non-Euclidean Tree-sitter DKB Knowledge Graphs

In the agent-native paradigm, source code does not exist as lines of text. The foundational layer of the static logic is formed by deterministic Abstract Syntax Tree (AST) derived knowledge graphs, referred to as the DKB.[4] Utilizing extraction algorithms akin to Tree-sitter, the paradigm captures pure structural and categorical representations, persisting them in a graph database capable of highly efficient bidirectional traversal.[4]

Unlike LLM-extracted knowledge graphs (LLM-KB), which are prone to hallucination and indexing incompleteness, the DKB deterministically constructs a code graph with absolute fidelity.[4] The pipeline injects labeled edges to denote inheritance, dependency injection patterns, and interface-consumer expansions, creating a mathematically rigorous topology that the LLM can navigate spatially using Graph Neural Networks.[4]

3.1 Mathematical Formalization of the Static Logic Graph

The codebase is formalized as an Attributed Directed Graph (ADG), enriched with categorical semantics derived directly from Homotopy Type Theory.[1] This approach treats logic not as a sequence of commands, but as a space of topological invariants.

Let the Static Logic Graph G_{static} be defined as a tuple:

$$G_{static} = (V, E, \phi_V, \phi_E, \mathcal{U}, \mathcal{M})$$

In this formalization, V represents the set of vertices, which are discrete, mechanically verifiable logical claims or operations (VPIR nodes). The set $E \subseteq V \times V$ represents the directed edges, denoting control flow, data dependencies, and structural relations such as extends or implements.[4] The attribute mapping functions $\phi_V : V \to \mathcal{P}(N)$ and

$\phi_E : E \to \mathcal{P}(N)$ assign semantic properties to nodes and edges. Crucially, $\mathcal{U}$ represents the univalent universe of Types in HoTT, ensuring that equivalent types can be treated as identical paths, and $\mathcal{M}$ represents the morphisms preserving the structural integrity between these types.[1]

By enforcing Typed Tokenization, natural language intent is parsed directly into objects, morphisms, paths, and types before being embedded into vectors.[1] This structured representation completely bypasses the ambiguities and syntax errors inherent in lexical parsing, allowing the LLM to operate directly on the semantic intermediate representation.

4. The Ephemeral Runtime: Dataflow Process Networks and Pipeline Algebra

If the Static Logic Graph constitutes the immutable laws of physics for the application, the dynamic runtime state is the energy traversing it. To completely eliminate the necessity of imperative loops—a construct that degrades LLM performance over extended contexts—the execution environment is modeled as a Dataflow Process Network (DPN), extending the mathematical principles of Kahn Process Networks.[7]

In the DPN architecture, the LLM instantiates independent "actors." These actors share no memory space. They communicate exclusively via non-blocking, unidirectional FIFO channels. When an actor requires data, it consumes a token from an input channel; upon computing a result, it yields a token to an output channel. State is never mutated in place; it flows as a continuous stream of immutable tokens, eliminating race conditions by design.

4.1 Categorical Semantics and Pipeline Algebra

The execution of the DPN is governed by Pipeline Algebra (PA), a category-theoretic workflow language.[6] PA treats every discrete step of computation—whether an LLM prompting node, a symbolic transformation, or a numeric calculation—as a typed, idempotent operator within a single algebraic expression.[6] Because operators are intrinsically idempotent and implemented via strict memoization, intermediate results are reused verbatim, yielding bit-level reproducibility regardless of the stochastic nature of the orchestrating LLM.[6]

The semantic mapping from the static logic to the dynamic execution is rigorously defined via categorical semantics. Let $\mathcal{P}$ be the category whose objects are Pipeline Algebra data types and whose morphisms are the pure functions defined in the Static Logic Graph.[6] Let $\mathcal{E}xec$ be the category of executable DPN nodes under composition.[6]

The runtime engine maps the static logic to the physical execution substrate via a semantics functor:

$$[-] : \mathcal{P} \longrightarrow \mathcal{E}xec$$

This functor maps each atomic operator to its runtime counterpart while preserving identities and composition. This preservation enables correctness-preserving optimizations, such as morphism fusion where $[\![Map(f) \circ Map(g)]\!] = [\![Map(f \circ g)]\!]$, and allows for dynamic parallel reordering across edge clusters without altering the deterministic outcome of the graph execution.[6]

Pipeline Algebra Concept	**DPN Runtime Implementation**	**Categorical Semantic Role**
Typed Operator	Stateless Actor Node	Morphism in Category $\mathcal{P}$
Data Flow	Unidirectional FIFO Queue	Object transition in $\mathcal{E}xec$
Idempotency	Memoized Token Verification	Preservation of Identity
Composition	Graph Topology Linking	Semantics Functor Mapping

5. Security and Mathematical Noninterference: The LLMbda Calculus

With the execution runtime modeled as a massively concurrent DPN, the system requires rigorous security guarantees against prompt injections and data contamination. Traditional Role-Based Access Control (RBAC) is fundamentally insufficient for stochastic environments where untrusted data can manipulate the reasoning engine itself. To solve this, the DPN runtime enforces dynamic Information-Flow Control (IFC) via the LLMbda Calculus.[3]

5.1 Big-Step Semantics and the Generation Operator

The LLMbda Calculus provides theoretical foundations for tracking security labels at the type-level during LLM execution.[13] The calculus extends the untyped call-by-value lambda

calculus with primitives specifically designed for LLM prompt-response manipulation.[3]

The core of the execution is formalized by a big-step semantics relation:

$$c, e \Downarrow c', v$$

This denotes that given an initial conversation context (or state history) c, the expression e evaluates to a value v and results in an updated context c'.[3]

The defining feature of this calculus is the generation operator, denoted as $@e$. This primitive serializes a value into a token sequence, dispatches it to the LLM, and parses the response back into a calculable term.[3] The formal semantic evaluation rule for this operator is defined as:

$$\frac{c, e \Downarrow c', v \quad \text{generate}(c' + \text{serialize}(v)) = r}{c, @e \Downarrow c' + [(\text{serialize}(v), r)], \text{parse}(r)}$$

Here, the LLM is modeled as a pure, deterministic function of the conversation history. The concatenation of the updated context c' and the serialized prompt yields a response r, which extends the conversation history and is parsed into the return value.[3]

5.2 Dynamic Information-Flow Control (IFC)

To prevent a malicious input from exfiltrating data or hijacking the agent, every token traversing a FIFO channel in the DPN is tagged with a security label from a join-semi-lattice $\mathcal{L}$.[3] The partial order operator $\sqsubseteq$ denotes the "may-flow-to" relation. For instance, if T represents Trusted data and U represents Untrusted data, the lattice enforces $T \sqsubseteq U$ (Trusted data may safely flow to an Untrusted context), but crucially rejects $U \sqsubseteq T$.

The calculus introduces primitives for attaching a security label to an expression ($l : e$) and for label testing ($l?e$). Label testing mechanically returns a boolean indicating if the label of e may safely flow to l.[3] The result of this test is semantically defined to be independent of the labels it inspects, allowing expressive policy enforcement without violating noninterference.[3]

5.3 The Noninterference Theorem

The foundational security guarantee of the LLMbda Calculus is encapsulated in Theorem 1:

Termination-Insensitive Noninterference.[3]

Theorem 1 (Termination-Insensitive Noninterference): *Ignoring termination behavior, information labeled at a security level* m *cannot influence observations at a level* n *unless* $m \sqsubseteq n$.

This theorem provides absolute mathematical integrity and confidentiality guarantees. If an actor in the DPN is infected by a prompt injection attack from an untrusted REST API, the IFC labeling explicitly prevents the output of that actor from flowing into any high-integrity system (such as a database write operation or a system shell command). The label constraints will fail at the mathematical level, terminating the localized data flow and quarantining the infection.[3]

6. The Legacy Interoperability Layer (FFI)

The agent-native environment, functioning strictly on pure HoTT logic graphs and DPN execution loops, must inevitably interface with the external, non-categorical world: Web2 REST APIs, legacy SQL databases, and procedural codebases. The Legacy Interoperability Layer—effectively the system's Foreign Function Interface (FFI)—bridges this gap without violating the LLMbda Calculus noninterference guarantees.

6.1 Epistemological Quarantining

To safely query a REST API, the runtime cannot simply execute an HTTP GET and map the response blindly into the DPN. Such an operation would corrupt the pure state and introduce untrusted vectors into the core logic. Instead, the FFI employs epistemological quarantining utilizing the explicit context management primitives of the LLMbda calculus: the fork and clear constructs.[3]

When an LLM orchestrator determines a legacy API call is required, it utilizes the fork e construct to spawn a temporary sub-conversation for the duration of the FFI interaction.[3] It then utilizes the clear command to explicitly wipe the sub-conversation history, quarantining the subsequent session from previous history.[3] This mechanism mathematically implements a "dual LLM pattern." A Privileged LLM (P-LLM) controls the core logic and generates verifiable code, while a Quarantined LLM (Q-LLM) interacts exclusively with the untrusted external API.[15] Because the FFI execution occurs within a clear boundary and is tagged with an untrusted security label (U), the external data cannot taint the P-LLM.

7. Phase 2: The Bridge Layer and Constrained Decoding Grammar

While the mathematical formalisms of HoTT, VPIR, and DPN provide a rigorous theoretical backend, standard autoregressive transformers natively output unstructured text tokens. The system requires typed graphs. To bridge this gap, the architecture mandates a Bridge Layer

utilizing a highly constrained decoding grammar. This grammar forces the standard LLM to output valid, mechanically verifiable VPIR nodes on day-one, preventing syntax hallucinations at the generation level.

7.1 Verifiable Programmatic Intermediate Representation (VPIR)

The Bridge Layer outputs logic in the form of VPIR. VPIR decouples logical construction from natural language rendering, ensuring mechanical verifiability of generated conditions.[16] In this framework, every logical step is forced into a structured JSON representation, defining a programmatic namespace $\mathcal{V}_t \triangleq \mathrm{keys}(F_t)$, which maps extracted facts to visual or semantic evidence.[17]

A VPIR layer is defined as a pair of executable predicate programs: the true-logic p_t and the counterfactual false-logic $\tilde{p}_t$.[17] The semantics of a VPIR predicate are strictly defined by its deterministic boolean output when evaluated in a sandboxed execution environment:

$$[\![p]\!](F_t) \triangleq \mathrm{Exec}(p; \mathrm{Env}(F_t)) \in \{0, 1\}$$

This programmatic formulation guarantees absolute verifiability. The generated predicates are accepted into the Static Logic Graph only if they pass mechanical execution against the fact space: $[\![p_t]\!](F_t) = 1$ and $[\![\tilde{p}_t]\!](F_t) = 0$.[17]

7.2 The Exact Bridge JSON Schema

To ensure that the autoregressive transformer exclusively emits valid VPIR nodes embedded with HoTT categorical types and LLMbda security labels, the generation is constrained by a strict JSON Schema Draft 2020-12 specification.[12] This schema acts as the impenetrable boundary between stochastic token generation and deterministic execution.

JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Agent-Native Bridge Layer: VPIR Node Schema",
  "description": "Constrained decoding grammar enforcing valid HoTT morphisms and VPIR predicates.",
  "type": "object",
  "properties": {
    "vpir_node_id": {
      "type": "string",
```

```
    "format": "uuid",
    "description": "Unique deterministic identifier for the graph node."
  },
  "hott_categorical_type": {
    "type": "object",
    "description": "Homotopy Type Theory definition mapping.",
    "properties": {
      "universe_level": {
        "type": "integer",
        "minimum": 0
      },
      "base_type": {
        "type": "string",
        "enum":
      },
      "path_induction_proof": {
        "type": "string",
        "description": "Optional serialized proof of topological invariants."
      }
    },
    "required": ["universe_level", "base_type"]
  },
  "vpir_predicate": {
    "type": "object",
    "description": "The executable predicate pair guaranteeing mechanical verifiability.",
    "properties": {
      "fact_namespace": {
        "type": "array",
        "items": { "type": "string" }
      },
      "true_logic_pt": {
        "type": "string",
        "description": "Executable true-logic mapping F_t to 1."
      },
      "false_logic_pt_tilde": {
        "type": "string",
        "description": "Counterfactual false-logic mapping F_t to 0."
      }
    },
    "required": ["fact_namespace", "true_logic_pt", "false_logic_pt_tilde"]
  },
  "smt_encoding": {
    "type": "object",
```

```
    "description": "Translation payload for the Z3 SMT solver.",
    "properties": {
      "logic_subset": {
        "type": "string",
        "enum":,
        "default": "QF_LIA"
      },
      "smt_lib_v2_assertions": {
        "type": "array",
        "items": {
          "type": "string",
          "pattern": "^\\(assert.*\\)$"
        }
      }
    },
    "required": ["logic_subset", "smt_lib_v2_assertions"]
  },
  "llmbda_security_routing": {
    "type": "object",
    "description": "Information-Flow Control tracking via the LLMbda calculus.",
    "properties": {
      "input_label": {
        "type": "string",
        "enum":
      },
      "output_label": {
        "type": "string",
        "enum":
      },
      "requires_fork_clear": {
        "type": "boolean",
        "description": "Flags if the node requires FFI epistemological quarantining."
      }
    },
    "required": ["input_label", "output_label", "requires_fork_clear"]
  }
},
"required": [
  "vpir_node_id",
  "hott_categorical_type",
  "vpir_predicate",
  "smt_encoding",
  "llmbda_security_routing"
```

```
]
}
```

Schema Component	Functional Purpose	Theoretical Backend
hott_categorical_type	Defines the topological shape and mathematical category of the node.	Homotopy Type Theory [9]
vpir_predicate	Ensures the logic is executable and strictly verifiable against facts.	VPIR Pipeline [17]
smt_encoding	Provides autoformalized first-order logic for constraint checking.	Satisfiability Modulo Theories [19]
llmbda_security_routing	Tags inputs/outputs for noninterference flow control and FFI isolation.	LLMbda Calculus [3]

8. Mathematical Formalization: VPIR to HoTT Morphisms

The Bridge Layer's JSON schema ensures syntactic correctness, but the semantic validity of the generated logic relies on a rigorous mathematical translation pipeline. The system must elevate the standard VPIR boolean predicates into categorical objects within the Homotopy Type Theory framework.

In HoTT, propositions are represented as types, and proofs as terms belonging to those types.

A VPIR true-logic predicate p_t operating on a fact namespace A is fundamentally a function mapping facts to booleans:

$$p_t : A \to \mathrm{Bool}$$

To formalize this within HoTT, we translate the VPIR predicate into a dependent type.

Specifically, the subset of elements in A for which the predicate holds true is modeled as a Σ -type (dependent pair type):

$$\sum_{x:A}(p_t(x) =_{\text{Bool}} \text{True})$$

A term of this type is a pair (x, prf) where x is a data token from the FIFO channel (an element of A) and prf is a proof that the VPIR predicate evaluates to True for x.

When an LLM attempts to define a transformation (a Pipeline Algebra operator) between two VPIR nodes, it is mathematically constructing a morphism between types. If node 1 is defined by type T_1 and node 2 by type T_2, the operation is a function $f : T_1 \to T_2$. The preservation of logical structure across this transformation relies on the Univalence Axiom, which states that equivalent types can be identified. If a neurosymbolic optimization refactors the execution graph, replacing f with an optimized path g, the system utilizes path induction to prove that f and g are homotopic:

$$\prod_{x:T_1} Id_{T_2}(f(x), g(x))$$

This topological invariant ensures that the LLM cannot hallucinate an optimization that alters the underlying deterministic behavior of the graph.

9. SMT Solvers and Neurosymbolic Refinement

While HoTT provides the overarching categorical structure, proving these topological invariants and dependent types in real-time is computationally intractable for raw univalent foundations. The execution engine must therefore autoformalize these atomic claims into first-order logic and evaluate them via SMT solvers (e.g., Z3) to guarantee constraint satisfaction at runtime.[20]

9.1 Mapping HoTT to SMT-LIB v2 Constraints

The translation from HoTT identity types to SMT-LIB v2 constraints forms the core of the Bridge Layer's verification pipeline. Because standard SMT solvers do not natively comprehend higher-order univalent universes, the mathematical translation pipeline projects HoTT singletons and equality types down into Quantifier-Free Linear Integer Arithmetic (QF_LIA) and algebraic datatypes supported by Z3.[19]

The translation pipeline operates as follows:

1. **Autoformalization:** The VPIR predicate p_t is extracted from the JSON output.
2. **Uninterpreted Functions:** The properties of the categorical object are defined as uninterpreted functions in SMT-LIB v2 to avoid introducing fresh variables.[24]

3. **Constraint Generation:** The Σ-type constraints are mapped to assert statements.

For example, if the VPIR predicate dictates that a value x must be positive and less than a dynamic threshold y retrieved from the FFI, the HoTT identity requirement is mapped to the following SMT-LIB v2 encoding within the JSON schema:

```lisp
(set-logic QF_LIA)
(declare-const x Int)
(declare-const y Int)
(assert (> x 0))
(assert (< x y))
(check-sat)
```

The execution engine delegates this payload to the Z3 solver. If the solver returns sat (Satisfiable), the runtime generates the necessary proof term prf, validating the Σ-type, and admits the token into the DPN FIFO channel.[19]

If the solver returns unsat—perhaps because an untrusted FFI Web2 API returned data violating the topological invariants—the token is immediately rejected. Crucially, the system does not crash. Instead, the orchestrator utilizes Active Inference to drive automated graph patching. By minimizing variational free energy (the divergence between expected constraints and actual observations), the system probabilistically generates an alternative routing solution, querying an alternative API or refining the prompt until the Z3 solver yields a sat result.

This deep integration of VPIR, HoTT morphisms, and SMT constraints guarantees that the LLM is physically incapable of executing hallucinated logic. The LLM simply proposes structural routing, and the mathematical runtime mechanically enforces reality.

Are you ready to proceed to Phase 3: The Shim MVP?

Works cited

1. (PDF) Running Category Theory and HoTT on Language: Typed Tokenization as a Software-First Path to Structured AI Reasoning - ResearchGate, accessed April 4, 2026, https://www.researchgate.net/publication/398355072_Running_Category_Theory_and_HoTT_on_Language_Typed_Tokenization_as_a_Software-First_Path_to_Structured_AI_Reasoning

2. 22076 PDFs | Review articles in TYPE THEORY - ResearchGate, accessed April 4, 2026, https://www.researchgate.net/topic/Type-Theory/publications
3. The LLMbda Calculus: AI Agents, Conversations, and ... - arXiv, accessed April 4, 2026, https://arxiv.org/abs/2602.20064
4. Reliable Graph-RAG for Codebases: AST-Derived Graphs vs LLM-Extracted Knowledge Graphs - arXiv, accessed April 4, 2026, https://arxiv.org/html/2601.08773v1
5. [2601.08773] Reliable Graph-RAG for Codebases: AST-Derived Graphs vs LLM-Extracted Knowledge Graphs - arXiv, accessed April 4, 2026, https://arxiv.org/abs/2601.08773
6. LLM-Powered, Expert-Refined Causal Loop Diagramming via Pipeline Algebra - MDPI, accessed April 4, 2026, https://www.mdpi.com/2079-8954/13/9/784
7. Hierarchical nodes in a dataflow process network may not be functional... - ResearchGate, accessed April 4, 2026, https://www.researchgate.net/figure/Hierarchical-nodes-in-a-dataflow-process-network-may-not-be-functional-even-if-the_fig2_260685566
8. cAST: Enhancing Code Retrieval-Augmented Generation with Structural Chunking via Abstract Syntax Tree - ResearchGate, accessed April 4, 2026, https://www.researchgate.net/publication/397421461_cAST_Enhancing_Code_Retrieval-Augmented_Generation_with_Structural_Chunking_via_Abstract_Syntax_Tree
9. QED at Large: A Survey of Engineering of Formally Verified Software | Foundations and Trends in Programming Languages - Emerald Insight, accessed April 4, 2026, https://www.emerald.com/ftpgl/article/5/2-3/102/1326570/QED-at-Large-A-Survey-of-Engineering-of-Formally
10. (PDF) Dataflow Process Networks - ResearchGate, accessed April 4, 2026, https://www.researchgate.net/publication/260685566_Dataflow_Process_Networks
11. (PDF) LLM-Powered, Expert-Refined Causal Loop Diagramming via Pipeline Algebra, accessed April 4, 2026, https://www.researchgate.net/publication/395501187_LLM-Powered_Expert-Refined_Causal_Loop_Diagramming_via_Pipeline_Algebra
12. Formal Analysis and Supply Chain Security for Agentic AI Skills - arXiv, accessed April 4, 2026, https://arxiv.org/html/2603.00195v1
13. Formal Analysis and Supply Chain Security for Agentic AI Skills - arXiv, accessed April 4, 2026, https://arxiv.org/pdf/2603.00195
14. COPLAS talk: The LLMbda Calculus: AI Agents, Conversations, and Information Flow - Københavns Universitet, accessed April 4, 2026, https://di.ku.dk/english/events/event-calendar-2026/coplas-talk-the-llmbda-calculus-ai-agents-conversations-and-information-flow/
15. The LLMbda Calculus: AI Agents, Conversations, and Information Flow - arXiv, accessed April 4, 2026, https://arxiv.org/pdf/2602.20064
16. Autonomous-Agents/README.md at main - GitHub, accessed April 4, 2026, https://github.com/tmgthb/Autonomous-Agents/blob/main/README.md
17. 2603.12266v1 | PDF | Formal Verification | Logic - Scribd, accessed April 4, 2026,

https://www.scribd.com/document/1011916332/2603-12266v1
18. MM-CondChain: A Programmatically Verified Benchmark for Visually Grounded Deep Compositional Reasoning - arXiv, accessed April 4, 2026, https://arxiv.org/html/2603.12266v1
19. Programming Z3 - Stanford CS Theory, accessed April 4, 2026, https://theory.stanford.edu/~nikolaj/programmingz3.html
20. Programming up to Congruence - Computer Science, accessed April 4, 2026, https://www.cs.yale.edu/homes/vilhelm/papers/popl15congruence.pdf
21. Formalizing Refinements and Constructive Algebra in Type Theory Anders M¨ortberg, accessed April 4, 2026, https://staff.math.su.se/anders.mortberg/thesis/thesis.pdf
22. Extensible Proof Engineering in Intensional Type Theory - Harvard DASH, accessed April 4, 2026, https://dash.harvard.edu/bitstreams/bf1a32d2-12dc-46e0-8f54-3398b506b4c4/download
23. HoTT-Crypt : A Study in Homotopy Type Theory based on Cryptography - EasyChair, accessed April 4, 2026, https://easychair.org/publications/paper/q2qC/open
24. Datatypes | Online Z3 Guide, accessed April 4, 2026, https://microsoft.github.io/z3guide/docs/theories/Datatypes/
25. Understanding how F* uses Z3 — Proof-Oriented Programming in F* documentation, accessed April 4, 2026, https://fstar-lang.org/tutorial/book/under_the_hood/uth_smt.html